moonprove 项目申报书

1 项目名称

moonprove — 把断言变成证明的 MoonBit 有界验证器

2 项目简介

moonprove 读入带契约的 MoonBit 风格代码，对每条 `assert` / `require` / `ensure` / `invariant` 给出三种判决之一：**证明成立**、**被反例推翻**、或**明确的证不出来。**

差别在第三种。多数检查器只会假设"进了 `if x < 0` 就说明下一行 `x >= 0`"，把这类假设堆成"没问题"的报告——断言真错了它也说没问题。moonprove 对每条义务取否命题交给求解器问"可满足吗"：不可满足就是证明，可满足就给出具体取值当反例，两头没结论就如实说证不出来。已发布 `NaIWaniubi/moonprove@0.2.0`，CLI 与浏览器 playground 是同一份代码编译到 native 与 wasm。

3 项目方向与通用性

方向：开发者工具 / 软件分析框架，具体是验证与求解。输入是纯文本规格，不绑定业务域与框架；表面语言自带词法与语法分析器（通用解析器会接受远超验证器能推理的范围）；输出标准 JSON 可接 CI；分析全在本地，无网络、无外部求解器进程。

4 预期使用场景

1. 1. **写不变量时当场验证**：加 `invariant i >= 0` 后立刻回答"进入时成立 / 保持成立 / 出口可用"三条各是证明还是反例，而不是等运行时崩。
2. 2. **CI 门禁区分"有问题"和"没搞定"**：退出码 3 专指"有义务未判定"，与 1（被推翻）分开，避免一次 `undefined` 被当成构建回归。
3. 3. **逐条复核判决**：`explain` 打印求解器自己的推理（是否走消元、收窄到哪个区间、在哪找到模型）。
4. 4. **审别人的断言**：代码贴进 playground，行号栏直接标出哪行被证、哪行被推翻、哪行证不出来。

5 拟实现的核心功能

- **判定过程四层流水线**：化简 → 区间反驳（利用整数离散性，判纯实数消元判不了的情形）→ Fourier–Motzkin 消元（有理数域的矛盾即整数域的矛盾）→ 边界搜索；
- **两个方向来自不同代码路径**：`Unsat`（证明）只来自前三者，`Sat`（反例）只来自重新校验过的模型，中间是 `Unknown`；
- **路径敏感符号执行**，循环用归纳而非展开；无 `invariant` 的循环得 `bounded` 且路径保守继续；
- **验证域显式声明**：每份报告打印 `domain:` 那一行，让证明与反例谈同一集合；
- **CLI**（`verify/explain/dump`）+ **浏览器 playground**；输出支持 text / JSON / SARIF 2.1.0，`violated` 直接进 GitHub code scanning 的 PR 标注；
- 质量证据：54 个单元测试（含与暴力枚举逐例对拍、SARIF 形状校验）、10 个语料样例判决与 `EXPECT` 行逐条核对、CLI 冒烟覆盖退出码与 JSON、浏览器 e2e 读 DOM 验证，全目标零警告。3,706 有效行是刻意的取舍：行数花在判定过程的可信度上。开发中测试抓出三个会导致**错误证明**的缺陷（`Nat` 无范围、32 位宽度溢出、系数抵消为 0 掩盖矛盾），均已修复并记入 `docs/DESIGN.md`。

6 原创性

原创，未移植任何现有代码。算法参考 Fourier–Motzkin 消元与 Hoare 逻辑的公开文献，实现为独立代码。

7 移植或参考说明

不适用（非移植）。唯一依赖 `moonbitlang/x` 0.5.1（Apache-2.0）；本项目 Apache-2.0。

8 GitHub 仓库链接

https://github.com/NaIWaniubi/moonprove —— 提交连续，CI 与 Pages 绿色，最新 `@0.2.0`。